UT2. MANEJO DE CONECTORES

ACTIVIDAD 3

Consulta SELECT sobre base de datos

Título: Consulta sobre base de datos.

Descripción:

En esta actividad, implementarás una clase ConsultaSELECT en Java que se conectará a una base de datos MySQL y ejecutará una consulta SELECT sobre una tabla llamada AGENTES. La consulta deberá recuperar y mostrar en la consola los valores de los campos CODIGO_AGENTE, NOMBRE_AGENTE, y FRASE_CLAVE para cada agente en la tabla.

Objetivos:

Desarrollar una aplicación en Java que se conecte a una base de datos MySQL y ejecute una consulta SELECT utilizando JDBC (Java Database Connectivity). Esta actividad te permitirá practicar la obtención de datos de una base de datos, el procesamiento de resultados y la gestión de excepciones SQL en Java.

Instrucciones:

1. **Implementación de la clase ConsultaSELECT:**
   - En tu proyecto Java, crea una clase llamada ConsultaSELECT.
   - Define las constantes necesarias (URL, USER, y PASSWORD) para gestionar la conexión a la base de datos **ut2**.

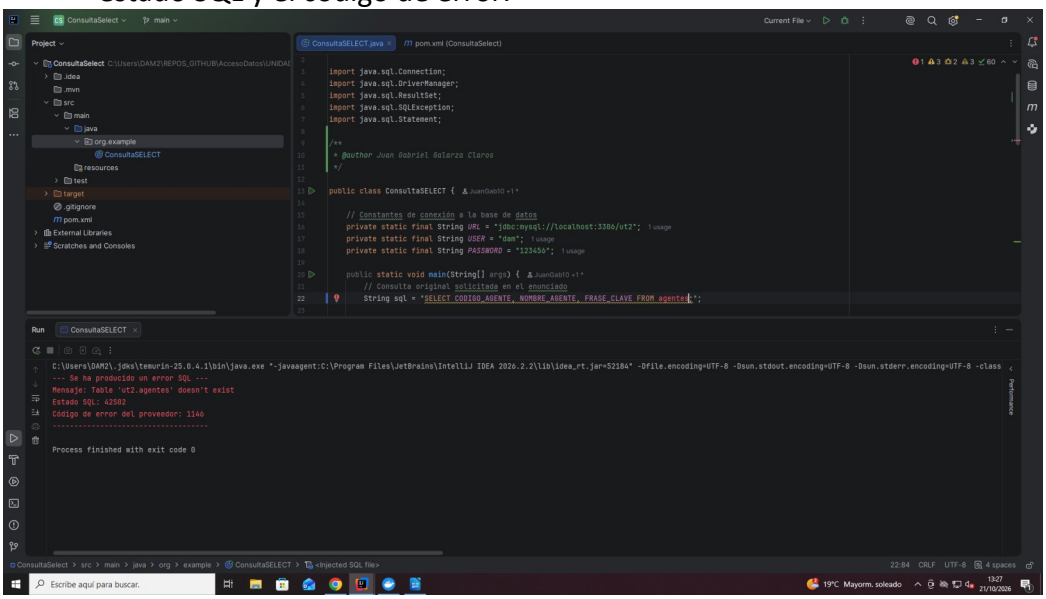
2. **Conectar a la base de datos:**
   - Utiliza un bloque try-with-resources para abrir la conexión, crear el Statement y obtener el ResultSet. Los tres se cerrarán automáticamente.

3. **Ejecutar la Consulta SELECT:**
   - Utiliza el objeto Statement para ejecutar la siguiente consulta SQL:
     SELECT CODIGO_AGENTE, NOMBRE_AGENTE, FRASE_CLAVE FROM AGENTES;
   - Esta consulta debe recuperar los datos de la tabla AGENTES y devolver un conjunto de resultados (ResultSet).

4. **Procesar los Resultados de la Consulta:**
   - Utiliza un bucle while para recorrer el ResultSet y obtener los valores de las columnas CODIGO_AGENTE, NOMBRE_AGENTE, y FRASE_CLAVE para cada fila.
   - Imprime cada registro en la consola con el siguiente formato:
     Código Agente: [CODIGO_AGENTE], Nombre Agente: [NOMBRE_AGENTE], Frase Clave: [FRASE_CLAVE]

5. **Gestión de Excepciones:**
   - Implementa un método llamado muestraErrorSQL que reciba un objeto SQLException y muestre en la consola: el mensaje de error (getMessage()), el estado SQL (getSQLState()) y el código de error del proveedor (getErrorCode()).
   - Captura la SQLException y llama a muestraErrorSQL pasándole la excepción.
   - Captura cualquier otra excepción y muestra el mensaje correspondiente en la consola.

6. **Prueba del programa:**
   - Ejecuta el programa y comprueba que se muestran los 9 agentes con el formato indicado.
   - Cambia en la consulta el nombre de la tabla por agentes (en minúsculas), vuelve a ejecutar y comprueba que muestraErrorSQL muestra el mensaje, el estado SQL y el código de error.



Rúbrica:

0	No se cumple ninguno de los requisitos mínimos.
3	Presenta la tarea, pero el código no es correcto.
5	Presenta la tarea, el código es correcto, pero no ha demostrado su funcionamiento mediante la captura de pantalla
7	Presenta la tarea, el código es correcto, demuestra su funcionamiento mediante la captura de pantalla, pero no describe los principales métodos incluidos en el código.
9	Se completa la tarea correctamente en todos sus apartados
10	Se implementa una funcionalidad extra o un detalle innovador que mejora el programa.

Entrega:

1. **Incluye aquí el código fuente**:

```
package org.example;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

/**
 * @author Juan Gabriel Galarza Claros
 */

public class ConsultaSELECT {

    // Constantes de conexión a la base de datos
    private static final String URL = "jdbc:mysql://localhost:3306/ut2";
    private static final String USER = "dam";
    private static final String PASSWORD = "123456";

    public static void main(String[] args) {
        // Consulta original solicitada en el enunciado
        String sql = "SELECT CODIGO_AGENTE, NOMBRE_AGENTE, FRASE_CLAVE FROM
AGENTES;";

        // Apertura de recursos con try-with-resources para su cierre automático
        try (Connection connection = DriverManager.getConnection(URL, USER, PASSWORD);
             Statement statement = connection.createStatement();
             ResultSet resultSet = statement.executeQuery(sql)) {

            int contadorAgentes = 0; // Funcionalidad extra: contador de registros procesados

            System.out.println("--- Listado de Agentes ---");

            // Recorrer el conjunto de resultados
            while (resultSet.next()) {
                int codigoAgente = resultSet.getInt("CODIGO_AGENTE");
                String nombreAgente = resultSet.getString("NOMBRE_AGENTE");
                String fraseClave = resultSet.getString("FRASE_CLAVE");

                // Formato de impresión exacto exigido en la tarea
                System.out.println("Código Agente: " + codigoAgente +
                        ", Nombre Agente: " + nombreAgente +
                        ", Frase Clave: " + fraseClave);

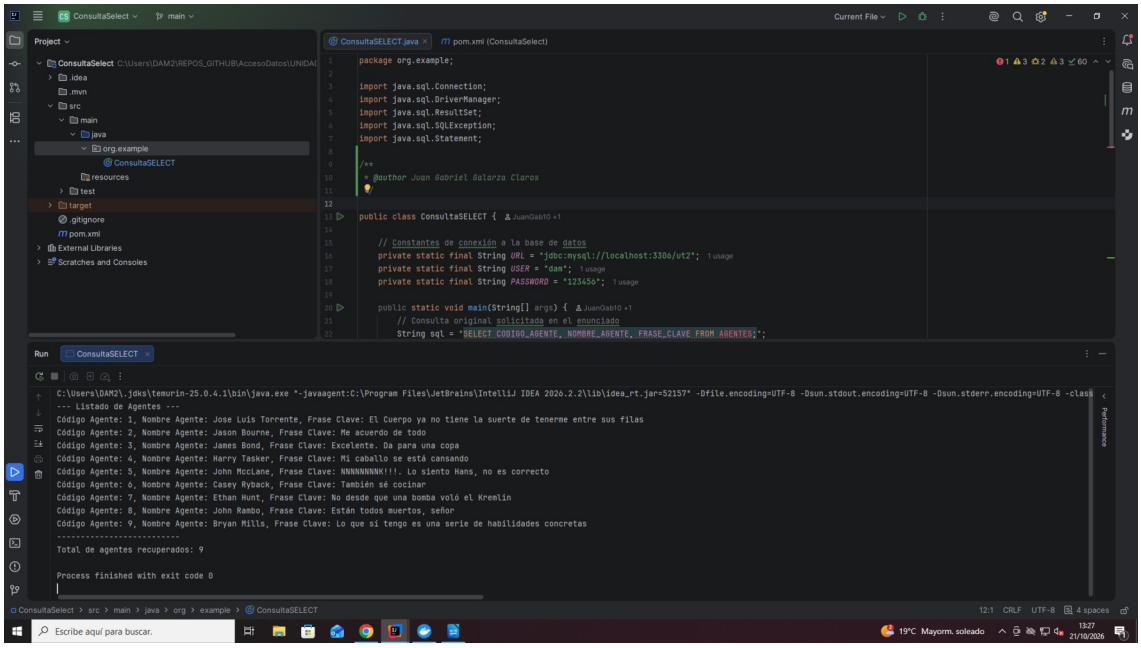
                contadorAgentes++; // Incremento del detalle extra
            }

            // Impresión del detalle innovador
            System.out.println("-------------------------");
            System.out.println("Total de agentes recuperados: " + contadorAgentes);
```

```
        } catch (SQLException e) {
            // Captura de excepción SQL y llamada al método de error personalizado
            muestraErrorSQL(e);
        } catch (Exception e) {
            // Captura de cualquier otra excepción no prevista
            System.err.println("Error inesperado: " + e.getMessage());
            e.printStackTrace(System.err);
        }
    }

    public static void muestraErrorSQL(SQLException e) {
        System.err.println("--- Se ha producido un error SQL ---");
        System.err.println("Mensaje: " + e.getMessage());
        System.err.println("Estado SQL: " + e.getSQLState());
        System.err.println("Código de error del proveedor: " + e.getErrorCode());
        System.err.println("-----------------------------------");
    }
}
```

2. **Incluye aquí las capturas de pantalla que demuestren el correcto funcionamiento del programa desarrollado**:



3. **Describe aquí los métodos y variables principales utilizadas en el programa**:

Clases Principales:

DriverManager: Clase utilizada para gestionar el conjunto de controladores JDBC y establecer la conexión con la base de datos MySQL mediante la URL facilitada.

Connection: Interfaz que mantiene y gestiona la sesión de conexión física activa con la base de datos ut2.

Statement: Interfaz que nos permite enviar la consulta de selección estática a la base de datos.

ResultSet: Objeto que representa la tabla de datos devuelta por la consulta, la cual incluye un cursor que hemos utilizado para iterar fila a fila.

SQLException: Clase que maneja y engloba cualquier error que ocurra durante el acceso a la base de datos.

Métodos Principales:

getConnection(): Establece la conexión validando la URL, USER y PASSWORD.

createStatement(): Genera el objeto necesario para poder lanzar sentencias SQL a través de la conexión activa.

executeQuery(): Ejecuta la consulta SQL estática y devuelve el objeto ResultSet con los datos requeridos.

next(): Desplaza el cursor del ResultSet a la siguiente fila. Devuelve true si hay registros, lo cual permite mantener activo el bucle while.

getInt() y getString(): Métodos encargados de extraer el valor de las columnas específicas de la fila actual del ResultSet.

muestraErrorSQL(SQLException e): Método para el manejo de errores.

Variables Principales:

URL, USER, PASSWORD: Constantes que almacenan los datos de configuración del servidor MySQL y las credenciales.

sql: Variable de tipo String que almacena la sentencia de la consulta SELECT .